Operating System-I Practical Slips

All 30 Slips — Question + Code Only

Slip numbers follow the 30-page practical-slip PDF. Repeated slips are retained. No sample outputs are included.

SLIP 1

Q.1

Write a C Menu driven Program to implement: a) Accept Available, b) Display Allocation, Max, c) Display the contents of need matrix, d) Display Available.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], finish[20] = {0}, safe[20];
    int count = 0, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available vector:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    while(count < n) {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                int possible = 1;
                for(j = 0; j < m; j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }
    if(count == n) {
        printf("\nSafe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("\nSystem is not in safe state.");

    }
    return 0;
}
```

Q.2

Write a C program that behaves like a shell which displays the command prompt '$'. It accepts the command, tokenize the command line and execute it by creating the child process. Also implement count c filename, count w filename and count l filename.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>
int main()
{
    char cmd[100], *args[20];
    int i;
    while(1) {
        printf("$ ");
        fgets(cmd,sizeof(cmd),stdin);
        cmd[strcspn(cmd,"\n")]=0;
        if(strcmp(cmd,"exit")==0)
            break;
        args[0]=strtok(cmd," ");
        i=0;
        while(args[i]!=NULL && i<19)
            args[++i]=strtok(NULL," ");
        if(args[0] && strcmp(args[0],"count")==0 && i>=2) {
            FILE *fp=fopen(args[2],"r");
            int ch, chars=0, words=0, lines=0, inword=0;
            if(fp==NULL) {
                printf("File cannot be opened.\n");
                continue;
```

```
            }
            while((ch=fgetc(fp))!=EOF) {
                chars++;
                if(ch=='\n') lines++;
                if(isspace(ch))
                    inword=0;
                else if(!inword) {
                    words++;
                    inword=1;
                }
            }
            fclose(fp);
            if(strcmp(args[1],"c")==0)
                printf("%d\n",chars);
            else if(strcmp(args[1],"w")==0)
                printf("%d\n",words);
            else if(strcmp(args[1],"l")==0)
                printf("%d\n",lines);
        } else {
            pid_t pid=fork();
            if(pid==0) {
                execvp(args[0],args);
                perror("Command failed");
                exit(1);
            } else if(pid>0) {
                wait(NULL);
            }
        }
    }
    return 0;
}
```

SLIP 2

Q.1

Write a program to simulate Linked file allocation method. Assume disk with n number of blocks. Randomly mark some blocks as allocated and maintain the list of free blocks. Implement Show Bit Vector, Create New File, Show Directory and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct File {
    char name[20];
    int blocks[100];
    int count;
};
int main()
{
    int n,bit[100],choice,i,j,b,freeCount;
    char name[20];
    struct File dir[20];
    int count=0;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i]=rand()%2;
    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name: ");
            scanf("%s",name);
            printf("Enter number of blocks: ");
            scanf("%d",&b);
            freeCount=0;
            for(i=0;i<n;i++)
                if(bit[i]==0) freeCount++;
            if(freeCount < b) {
                printf("Not enough free blocks.");
            } else {
                strcpy(dir[count].name,name);
                dir[count].count=0;
                for(i=0;i<n && dir[count].count<b;i++) {
                    if(bit[i]==0) {
                        bit[i]=1;
                        dir[count].blocks[dir[count].count++]=i;
                    }
                }
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tBlocks\n");
            for(i=0;i<count;i++) {
                printf("%s\t",dir[i].name);
                for(j=0;j<dir[i].count;j++)
                    printf("%d ",dir[i].blocks[j]);
                printf("\n");
            }
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);
            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].count;j++)
                        bit[dir[i].blocks[j]]=0;

                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];
                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using FCFS algorithm. Accept total number of disk blocks, disk request string and current head position. Display the list of requests in the order in which it is served and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
int main()
```

```
{
    int n,i,head,total=0;
    int req[100];
    printf("Enter number of disk requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter current head position: ");
    scanf("%d",&head);
    printf("\nService order: %d ",head);
    for(i=0;i<n;i++) {
        total += abs(head-req[i]);
        head = req[i];
        printf("%d ",head);
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 3

Q.1

Write a C program to simulate Banker's algorithm for deadlock avoidance. Calculate and display the Need matrix and check whether the system is in a safe state; if safe, display the safe sequence.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], finish[20] = {0}, safe[20];
    int count = 0, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available vector:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    while(count < n) {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                int possible = 1;
                for(j = 0; j < m; j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }
    if(count == n) {
        printf("\nSafe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("\nSystem is not in safe state.");

    }
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using SSTF algorithm. Accept disk request string and current head position. Display the requests in service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n,i,j,head,total=0,index,diff,min;
    int req[100],done[100]={0};
    printf("Enter number of disk requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter current head position: ");
    scanf("%d",&head);
    printf("\nService order: ");
    for(i=0;i<n;i++) {
        min=100000;
        index=-1;
        for(j=0;j<n;j++) {
            if(!done[j]) {
                diff=abs(head-req[j]);
                if(diff<min) {
                    min=diff;
                    index=j;
                }
            }
        }
```

```
        done[index]=1;
        total += abs(head-req[index]);
        head=req[index];
        printf("%d ",head);
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 4

Q.1

Implement the Menu driven Banker's algorithm for accepting Allocation and Max from user: a) Accept Available, b) Display Allocation, Max, c) Find Need and display it, d) Display Available.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], finish[20] = {0}, safe[20];
    int count = 0, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available vector:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    while(count < n) {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                int possible = 1;
                for(j = 0; j < m; j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }
    if(count == n) {
        printf("\nSafe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("\nSystem is not in safe state.");

    }
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using SCAN algorithm. Accept total disk blocks, disk request string and current head position. Display the requests in service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,size,head,i,split,total=0;
    int req[100];
    printf("Enter disk size: ");
    scanf("%d",&size);
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    sort(req,n);
    printf("Enter direction (0=Left, 1=Right): ");
```

```
    scanf("%d",&split);
    printf("\nService order: ");
    if(split==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += abs(head-(size-1));
        head=size-1;
        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += abs(head-0);
        head=0;
        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 5

Q.1

Write the simulation program for demand paging and show page scheduling and total number of page faults according to the FIFO page replacement algorithm. Assume memory of n frames. Reference String: 3,4,5,6,3,4,7,3,4,5,6,7,2,4,6.

```
#include <stdio.h>
int main()
{
    int p[50], f[20];
    int n, nf, i, j, pos = 0;
    int hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
        f[i] = -1;
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            f[pos] = p[i];
            pos = (pos + 1) % nf;
            fault++;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

Q.2

Consider a system with n processes and m resource types. Accept instances, allocation and maximum requirement matrices. Display the Need matrix and check if a given request of a process can be granted immediately using the Resource Request algorithm.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, p;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], req[20], finish[20] = {0};
    int safe[20], count = 0, possible, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);
    printf("Enter Available:\n");
    for(j=0;j<m;j++)
        scanf("%d",&avail[j]);
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("Enter process number for request: ");
    scanf("%d",&p);
    printf("Enter request vector:\n");
    for(j=0;j<m;j++)
        scanf("%d",&req[j]);
    for(j=0;j<m;j++) {
        if(req[j] > need[p][j] || req[j] > avail[j]) {
            printf("Request cannot be granted immediately.");
            return 0;
        }
    }
    for(j=0;j<m;j++) {
        avail[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
        work[j] = avail[j];
    }
    while(count < n) {
        found = 0;
        for(i=0;i<n;i++) {
            if(!finish[i]) {
                possible = 1;
                for(j=0;j<m;j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j=0;j<m;j++)
                        work[j] += alloc[i][j];
                    finish[i] = 1;
                    safe[count++] = i;
                    found = 1;
                }
```

```
            }
        }
        if(!found) break;
    }
    if(count == n) {
        printf("Request can be granted.");
        printf("\nSafe sequence: ");
        for(i=0;i<n;i++)
            printf("P%d ",safe[i]);
    } else {
        printf("Request cannot be granted safely.");
    }

    return 0;
}
```

SLIP 6

Q.1

Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n blocks. Randomly mark some blocks as allocated and maintain the list of free blocks. Implement Show Bit Vector, Create New File, Show Directory and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct File {
    char name[20];
    int start;
    int length;
};
int main()
{
    int n, bit[100], choice, i, j, start, len, ok;
    char name[20];
    struct File dir[20];
    int count=0;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i] = rand()%2;
    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name and length: ");
            scanf("%s%d",name,&len);
            start=-1;
            for(i=0;i<=n-len;i++) {
                ok=1;
                for(j=0;j<len;j++)
                    if(bit[i+j]) ok=0;
                if(ok) {
                    start=i;
                    break;
                }
            }
            if(start==-1) {
                printf("File cannot be allocated.");
            } else {
                for(j=0;j<len;j++)
                    bit[start+j]=1;
                strcpy(dir[count].name,name);
                dir[count].start=start;
                dir[count].length=len;
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tStart\tLength\n");
            for(i=0;i<count;i++)
                printf("%s\t%d\t%d\n",
                       dir[i].name,dir[i].start,dir[i].length);
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);
            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].length;j++)
                        bit[dir[i].start+j]=0;

                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];
                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using C-SCAN algorithm. Accept total number of disk blocks, disk request string and current head position. Display the requests in service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
```

```
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,size,head,direction,i,total=0;
    int req[100];
    printf("Enter disk size: ");
    scanf("%d",&size);
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);
    sort(req,n);
    printf("\nService order: ");
    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += abs(head-(size-1));
        head=size-1;
        total += size-1;
        head=0;
        for(i=0;i<n;i++)
            if(req[i]<head || req[i]>=0) {
                if(req[i] < size && req[i] < req[0]) {
                    total += abs(head-req[i]);
                    head=req[i];
                    printf("%d ",head);
                }
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += head;
        head=0;
        total += size-1;
        head=size-1;
        for(i=n-1;i>=0;i--)
            if(req[i]>req[n-1]) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }

    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 7

Q.1

Consider the given system snapshot. Using the Resource-Request algorithm, check whether the current system is in safe state or not.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, p;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], req[20], finish[20] = {0};
    int safe[20], count = 0, possible, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);
    printf("Enter Available:\n");
    for(j=0;j<m;j++)
        scanf("%d",&avail[j]);
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("Enter process number for request: ");
    scanf("%d",&p);
    printf("Enter request vector:\n");
    for(j=0;j<m;j++)
        scanf("%d",&req[j]);
    for(j=0;j<m;j++) {
        if(req[j] > need[p][j] || req[j] > avail[j]) {
            printf("Request cannot be granted immediately.");
            return 0;
        }
    }
    for(j=0;j<m;j++) {
        avail[j] -= req[j];
        alloc[p][j] += req[j];
        need[p][j] -= req[j];
        work[j] = avail[j];
    }
    while(count < n) {
        found = 0;
        for(i=0;i<n;i++) {
            if(!finish[i]) {
                possible = 1;
                for(j=0;j<m;j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j=0;j<m;j++)
                        work[j] += alloc[i][j];
                    finish[i] = 1;
                    safe[count++] = i;
                    found = 1;
                }
            }
        }
        if(!found) break;
    }
    if(count == n) {
        printf("Request can be granted.");
        printf("\nSafe sequence: ");
        for(i=0;i<n;i++)
            printf("P%d ",safe[i]);
    } else {
        printf("Request cannot be granted safely.");
    }

    return 0;
}
```

Q.2

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Child priority before nice = %d\n",nice(0));
        if(nice(-5) == -1)
            perror("nice");
        printf("Child priority after nice request = %d\n",nice(0));
```

```
    } else {
        wait(NULL);
        printf("Parent completed.\n");
    }
    return 0;
}
```

SLIP 8

Q.1

Write a program to simulate Index file allocation method. Assume disk with n blocks. Randomly mark some blocks as allocated and maintain the list of free blocks. Implement Show Bit Vector, Create New File, Show Directory and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct File {
    char name[20];
    int indexBlock;
    int blocks[100];
    int count;
};
int main()
{
    int n,bit[100],choice,i,j,b,freeCount,indexBlock;
    char name[20];
    struct File dir[20];
    int count=0;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i]=rand()%2;
    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name and number of data blocks: ");
            scanf("%s%d",name,&b);
            indexBlock=-1;
            for(i=0;i<n;i++)
                if(bit[i]==0) {
                    indexBlock=i;
                    break;
                }
            if(indexBlock==-1) {
                printf("No free index block.");
                continue;
            }
            freeCount=0;
            for(i=0;i<n;i++)
                if(bit[i]==0 && i!=indexBlock)
                    freeCount++;
            if(freeCount<b) {
                printf("Not enough free blocks.");
                continue;
            }
            bit[indexBlock]=1;
            strcpy(dir[count].name,name);
            dir[count].indexBlock=indexBlock;
            dir[count].count=0;
            for(i=0;i<n && dir[count].count<b;i++) {
                if(bit[i]==0) {
                    bit[i]=1;
                    dir[count].blocks[dir[count].count++]=i;
                }
            }
            count++;
            printf("File allocated.");
        }
        else if(choice==3) {

            printf("\nFile\tIndex\tData Blocks\n");
            for(i=0;i<count;i++) {
                printf("%s\t%d\t",dir[i].name,dir[i].indexBlock);
                for(j=0;j<dir[i].count;j++)
                    printf("%d ",dir[i].blocks[j]);
                printf("\n");
            }
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);
            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    bit[dir[i].indexBlock]=0;
                    for(j=0;j<dir[i].count;j++)
                        bit[dir[i].blocks[j]]=0;
                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];
                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }
    return 0;
```

```
}
```

Q.2

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Child priority before nice = %d\n",nice(0));
        if(nice(-5) == -1)
            perror("nice");
        printf("Child priority after nice request = %d\n",nice(0));
    } else {
        wait(NULL);
        printf("Parent completed.\n");
    }
    return 0;
}
```

SLIP 9

Q.1

Write the simulation program for demand paging and show page scheduling and total number of page faults according to the Optimal page replacement algorithm. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20];
    int n, nf, i, j, k, pos, far, next;
    int hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
        f[i] = -1;
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                far = -1;
                for(j=0;j<nf;j++) {
                    next = n + 1;
                    for(k=i+1;k<n;k++) {
                        if(f[j] == p[k]) {
                            next = k;
                            break;
                        }
                    }
                    if(next > far) {
                        far = next;
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

Q.2

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. Accept arrival time and first CPU burst for n processes; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time=0, done=0, idx, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20], used[20]={0};
    float awt=0, atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    printf("\nGantt Chart:\n");
    while(done<n) {
        idx=-1;
        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || bt[i]<bt[idx])
                    idx=i;
        if(idx==-1) {
            time++;
            continue;
        }
        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");
```

```
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

SLIP 10

Q.1

Write the program to simulate Round Robin (RR) scheduling. Accept arrival time, first CPU burst and time quantum; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n,tq,i,j,time=0,completed=0;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];
    float awt=0,atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d",&tq);
    while(completed<n) {
        int executed=0;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                executed=1;
                printf("| P%d ",pid[i]);
                if(rem[i]>tq) {
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    rem[i]=0;
                    ct[i]=time;
                    tat[i]=ct[i]-at[i];
                    wt[i]=tat[i]-bt[i];
                    completed++;
                }
            }
        }
        if(!executed)
            time++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using LOOK algorithm. Accept disk blocks, request string and current head position. Display service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,head,direction,i,total=0;
    int req[100];
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);
    sort(req,n);
    printf("\nService order: ");
    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
```

```
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 11

Q.1

Write a C program to simulate Banker's algorithm for deadlock avoidance. Implement a menu to Accept Available, Display Allocation and Max, Display Need matrix and Display Available.

```
#include <stdio.h>
int main()
{
    int n, m, i, j, k;
    int alloc[20][20], max[20][20], need[20][20];
    int avail[20], work[20], finish[20] = {0}, safe[20];
    int count = 0, found;
    printf("Enter number of processes and resources: ");
    scanf("%d%d", &n, &m);
    printf("Enter Allocation matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);
    printf("Enter Max matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);
    printf("Enter Available vector:\n");
    for(j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
        work[j] = avail[j];
    }
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    printf("\nNeed Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < m; j++)
            printf("%d ", need[i][j]);
        printf("\n");
    }
    while(count < n) {
        found = 0;
        for(i = 0; i < n; i++) {
            if(!finish[i]) {
                int possible = 1;
                for(j = 0; j < m; j++)
                    if(need[i][j] > work[j])
                        possible = 0;
                if(possible) {
                    for(j = 0; j < m; j++)
                        work[j] += alloc[i][j];
                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(!found)
            break;
    }
    if(count == n) {
        printf("\nSafe Sequence: ");
        for(i = 0; i < n; i++)
            printf("P%d ", safe[i]);
    } else {
        printf("\nSystem is not in safe state.");

    }
    return 0;
}
```

Q.2

Write the simulation program for demand paging and show page scheduling and total page faults according to the Optimal page replacement algorithm. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20];
    int n, nf, i, j, k, pos, far, next;
    int hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
        f[i] = -1;
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
```

```
                    break;
                }
            }
            if(pos == -1) {
                far = -1;
                for(j=0;j<nf;j++) {
                    next = n + 1;
                    for(k=i+1;k<n;k++) {
                        if(f[j] == p[k]) {
                            next = k;
                            break;
                        }
                    }
                    if(next > far) {
                        far = next;
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 12

Q.1

Write the program to simulate FCFS CPU scheduling. Accept arrival time and first CPU burst for n processes; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time = 0, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i] = i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(at[i] > at[j]) {
                temp=at[i]; at[i]=at[j]; at[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=pid[i]; pid[i]=pid[j]; pid[j]=temp;
            }
    printf("\nGantt Chart:\n");
    for(i=0;i<n;i++) {
        if(time < at[i])
            time = at[i];
        printf("| P%d ",pid[i]);
        time += bt[i];
        ct[i] = time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

Q.2

Implement a C program to accept n integers. Main creates a child using fork(). Parent sorts using bubble sort and waits using wait(); child sorts using insertion sort.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void bubble(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]) {
                t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
}
void insertion(int a[], int n)
{
    int i,j,key;
    for(i=1;i<n;i++) {
        key=a[i];
        j=i-1;
        while(j>=0 && a[j]>key) {
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}
void display(int a[], int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\n");
}
int main()
{
    int n,i;
    int a[50],b[50];
    pid_t pid;
    printf("Enter number of integers: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        scanf("%d",&a[i]);
        b[i]=a[i];
    }
    pid=fork();
    if(pid<0) {
        perror("fork");
        return 1;
    }
    if(pid==0) {
        insertion(b,n);
```

```
        printf("Child sorted using insertion sort: ");
        display(b,n);
    } else {
        bubble(a,n);
        printf("Parent sorted using bubble sort: ");
        display(a,n);
        wait(NULL);
    }
    return 0;
}
```

SLIP 13

Q.1

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. Accept arrival time and first CPU burst; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time=0, done=0, idx, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20], used[20]={0};
    float awt=0, atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    printf("\nGantt Chart:\n");
    while(done<n) {
        idx=-1;
        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || bt[i]<bt[idx])
                    idx=i;
        if(idx==-1) {
            time++;
            continue;
        }
        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

Q.2

Write a simulation program for disk scheduling using LOOK algorithm. Accept disk blocks, request string and current head position. Display service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,head,direction,i,total=0;
    int req[100];
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);
    sort(req,n);
    printf("\nService order: ");
    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
```

```
                head=req[i];
                printf("%d ",head);
            }
        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

SLIP 14

Q.1

Implement the C program that accepts an integer array. Main forks a child process. Parent sorts the array and passes the sorted array to child through command line arguments of execve(). Child loads a new program using execve() to perform binary search.

```
/* parent.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,i,key;
    int a[50];
    char *args[60];
    printf("Enter number of integers: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter item to search: ");
    scanf("%d",&key);
    sort(a,n);
    args[0]="./child";
    for(i=0;i<n;i++)
        args[i+1]=(char*)malloc(12);
    for(i=0;i<n;i++)
        sprintf(args[i+1],"%d",a[i]);
    args[n+1]=(char*)malloc(12);
    sprintf(args[n+1],"%d",key);
    args[n+2]=NULL;
    if(fork()==0) {
        execve("./child",args,NULL);
        perror("execve");
        exit(1);
    }
    wait(NULL);
    return 0;
}
/* child.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[])
{
    int n=argc-2, key=atoi(argv[argc-1]);
    int low=0, high=n-1, mid, i;
    while(low<=high) {
        mid=(low+high)/2;
        if(atoi(argv[mid+1])==key) {
            printf("Item found.\n");
            return 0;
        }
        if(atoi(argv[mid+1])<key)
            low=mid+1;
        else
            high=mid-1;
    }
    printf("Item not found.\n");
    return 0;

}
```

Q.2

Write the simulation program for demand paging and show page scheduling and total page faults according to FIFO page replacement. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20];
    int n, nf, i, j, pos = 0;
    int hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
        f[i] = -1;
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            f[pos] = p[i];
            pos = (pos + 1) % nf;
```

```
            fault++;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 15

Q.1

Write a C program to illustrate the concept of orphan process. Parent creates a child and terminates before child finishes, so child becomes orphan. Use fork(), sleep(), getpid(), getppid().

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Initial Parent PID = %d\n",getppid());
        sleep(5);
        printf("Parent PID after parent termination = %d\n",getppid());
    } else {
        printf("Parent PID = %d\n",getpid());
        sleep(1);
        printf("Parent terminating...\n");
        exit(0);
    }
    return 0;
}
```

Q.2

Write the program to simulate Preemptive Priority scheduling. Accept arrival time, first CPU burst and priority for n processes; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n,i,time=0,completed=0,idx;
    int pid[20],at[20],bt[20],pr[20],rem[20],ct[20],tat[20],wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rem[i]=bt[i];
    }
    while(completed<n) {
        idx=-1;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;
            }
        }
        if(idx==-1) {
            time++;
            continue;
        }
        rem[idx]--;
        time++;
        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

SLIP 16

Q.1

Write the simulation program to implement demand paging and show page scheduling and total page faults according to LRU using counter method. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], counter[20];
    int n, nf, i, j, pos, min, hit, fault = 0;
    int time = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        counter[i] = 0;
    }
    for(i=0;i<n;i++) {
        time++;
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                hit = 1;
                counter[j] = time;
                break;
            }
        }
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                min = counter[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(counter[j] < min) {
                        min = counter[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            counter[pos] = time;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

Q.2

Write a C program that behaves like a shell displaying '$'. Tokenize and execute commands using a child process and implement count c filename, count w filename and count l filename.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>
int main()
{
    char cmd[100], *args[20];
    int i;
    while(1) {
        printf("$ ");
        fgets(cmd,sizeof(cmd),stdin);
        cmd[strcspn(cmd,"\n")]=0;
        if(strcmp(cmd,"exit")==0)
            break;
        args[0]=strtok(cmd," ");
        i=0;
        while(args[i]!=NULL && i<19)
            args[++i]=strtok(NULL," ");
        if(args[0] && strcmp(args[0],"count")==0 && i>=2) {
            FILE *fp=fopen(args[2],"r");
            int ch, chars=0, words=0, lines=0, inword=0;
            if(fp==NULL) {
                printf("File cannot be opened.\n");
                continue;
            }
            while((ch=fgetc(fp))!=EOF) {
                chars++;
                if(ch=='\n') lines++;
                if(isspace(ch))
```

```
                inword=0;
            else if(!inword) {
                words++;
                inword=1;
            }
        }
        fclose(fp);
        if(strcmp(args[1],"c")==0)
            printf("%d\n",chars);
        else if(strcmp(args[1],"w")==0)
            printf("%d\n",words);
        else if(strcmp(args[1],"l")==0)
            printf("%d\n",lines);
    } else {
        pid_t pid=fork();
        if(pid==0) {
            execvp(args[0],args);
            perror("Command failed");
            exit(1);
        } else if(pid>0) {
            wait(NULL);
        }
    }
}
return 0;
}
```

SLIP 17

Q.1

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. Accept arrival time and first CPU burst; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time=0, done=0, idx, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20], used[20]={0};
    float awt=0, atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    printf("\nGantt Chart:\n");
    while(done<n) {
        idx=-1;
        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || bt[i]<bt[idx])
                    idx=i;
        if(idx==-1) {
            time++;
            continue;
        }
        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

Q.2

Write a program to simulate Sequential (Contiguous) file allocation method. Assume disk with n blocks; randomly mark blocks allocated and maintain free blocks. Implement Show Bit Vector, Create New File, Show Directory, Delete File and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct File {
    char name[20];
    int start;
    int length;
};
int main()
{
    int n, bit[100], choice, i, j, start, len, ok;
    char name[20];
    struct File dir[20];
    int count=0;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i] = rand()%2;
    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name and length: ");
            scanf("%s%d",name,&len);
            start=-1;
            for(i=0;i<=n-len;i++) {
                ok=1;
                for(j=0;j<len;j++)
                    if(bit[i+j]) ok=0;
                if(ok) {
                    start=i;
                    break;
                }
```

```
            }
            if(start==-1) {
                printf("File cannot be allocated.");
            } else {
                for(j=0;j<len;j++)
                    bit[start+j]=1;
                strcpy(dir[count].name,name);
                dir[count].start=start;
                dir[count].length=len;
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tStart\tLength\n");
            for(i=0;i<count;i++)
                printf("%s\t%d\t%d\n",
                       dir[i].name,dir[i].start,dir[i].length);
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);
            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].length;j++)
                        bit[dir[i].start+j]=0;

                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];
                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }
    return 0;
}
```

SLIP 18

Q.1

Write the program to simulate Preemptive Shortest Job First (SJF) scheduling. Accept arrival time and first CPU burst; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n,i,time=0,completed=0,idx,min;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }
    while(completed<n) {
        idx=-1;
        min=100000;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0 && rem[i]<min) {
                min=rem[i];
                idx=i;
            }
        }
        if(idx==-1) {
            time++;
            continue;
        }
        rem[idx]--;
        time++;
        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

Q.2

Write a C program to illustrate the concept of orphan process. Parent creates a child and terminates before child finishes. Use fork(), sleep(), getpid(), getppid().

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Initial Parent PID = %d\n",getppid());
        sleep(5);
        printf("Parent PID after parent termination = %d\n",getppid());
    } else {
        printf("Parent PID = %d\n",getpid());
        sleep(1);
        printf("Parent terminating...\n");
        exit(0);
    }
    return 0;
}
```

SLIP 19

Q.1

Write the program to simulate Preemptive Priority scheduling. Accept arrival time, first CPU burst and priority for n processes; generate the next CPU burst.

```
#include <stdio.h>
int main()
{
    int n,i,time=0,completed=0,idx;
    int pid[20],at[20],bt[20],pr[20],rem[20],ct[20],tat[20],wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rem[i]=bt[i];
    }
    while(completed<n) {
        idx=-1;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;
            }
        }
        if(idx==-1) {
            time++;
            continue;
        }
        rem[idx]--;
        time++;
        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

Q.2

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Child priority before nice = %d\n",nice(0));
        if(nice(-5) == -1)
            perror("nice");
        printf("Child priority after nice request = %d\n",nice(0));
    } else {
        wait(NULL);
        printf("Parent completed.\n");
    }
    return 0;
}
```

SLIP 20

Q.1

Write a simulation program for disk scheduling using LOOK algorithm. Accept disk blocks, request string and current head position. Display service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,head,direction,i,total=0;
    int req[100];
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&direction);
    sort(req,n);
    printf("\nService order: ");
    if(direction==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

Q.2

Write the simulation program to implement demand paging and show page scheduling and total page faults according to LRU using counter method. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], counter[20];
    int n, nf, i, j, pos, min, hit, fault = 0;
    int time = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        counter[i] = 0;
    }
    for(i=0;i<n;i++) {
        time++;
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                hit = 1;
                counter[j] = time;
                break;
            }
        }
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
```

```
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                min = counter[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(counter[j] < min) {
                        min = counter[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            counter[pos] = time;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 21

Q.1

Write the simulation program for demand paging and show page scheduling and total page faults according to MFU page replacement. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], freq[20];
    int n, nf, i, j, pos, max, hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        freq[i] = 0;
    }
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                freq[j]++;
                hit = 1;
                break;
            }
        }
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                max = freq[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(freq[j] > max) {
                        max = freq[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            freq[pos] = 1;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

Q.2

Write a program that demonstrates the use of nice() system call. After a child process is started using fork(), assign higher priority to the child using nice() system call.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main()
{
    pid_t pid;
    pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Child priority before nice = %d\n",nice(0));
        if(nice(-5) == -1)
            perror("nice");
        printf("Child priority after nice request = %d\n",nice(0));
    } else {
        wait(NULL);
        printf("Parent completed.\n");
    }
    return 0;
}
```

SLIP 22

Q.1

Write a program to simulate Linked file allocation method. Assume disk with n blocks. Randomly mark blocks allocated and maintain free blocks. Implement Show Bit Vector, Create New File, Show Directory, Delete File and Exit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
struct File {
    char name[20];
    int blocks[100];
    int count;
};
int main()
{
    int n,bit[100],choice,i,j,b,freeCount;
    char name[20];
    struct File dir[20];
    int count=0;
    printf("Enter number of disk blocks: ");
    scanf("%d",&n);
    srand(time(NULL));
    for(i=0;i<n;i++)
        bit[i]=rand()%2;
    while(1) {
        printf("\n1.Show Bit Vector");
        printf("\n2.Create New File");
        printf("\n3.Show Directory");
        printf("\n4.Delete File");
        printf("\n5.Exit");
        printf("\nEnter choice: ");
        scanf("%d",&choice);
        if(choice==1) {
            for(i=0;i<n;i++) printf("%d ",bit[i]);
        }
        else if(choice==2) {
            printf("Enter file name: ");
            scanf("%s",name);
            printf("Enter number of blocks: ");
            scanf("%d",&b);
            freeCount=0;
            for(i=0;i<n;i++)
                if(bit[i]==0) freeCount++;
            if(freeCount < b) {
                printf("Not enough free blocks.");
            } else {
                strcpy(dir[count].name,name);
                dir[count].count=0;
                for(i=0;i<n && dir[count].count<b;i++) {
                    if(bit[i]==0) {
                        bit[i]=1;
                        dir[count].blocks[dir[count].count++]=i;
                    }
                }
                count++;
                printf("File allocated.");
            }
        }
        else if(choice==3) {
            printf("\nFile\tBlocks\n");
            for(i=0;i<count;i++) {
                printf("%s\t",dir[i].name);
                for(j=0;j<dir[i].count;j++)
                    printf("%d ",dir[i].blocks[j]);
                printf("\n");
            }
        }
        else if(choice==4) {
            printf("Enter file name: ");
            scanf("%s",name);
            for(i=0;i<count;i++) {
                if(strcmp(dir[i].name,name)==0) {
                    for(j=0;j<dir[i].count;j++)
                        bit[dir[i].blocks[j]]=0;

                    for(j=i;j<count-1;j++)
                        dir[j]=dir[j+1];
                    count--;
                    printf("File deleted.");
                    break;
                }
            }
        }
        else if(choice==5)
            break;
    }
    return 0;
}
```

Q.2

Write the program to simulate Non-preemptive Priority scheduling. Accept arrival time and first CPU burst; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
```

```
    int n,i,time=0,done=0,idx;
    int pid[20],at[20],bt[20],pr[20],ct[20],tat[20],wt[20],used[20]={0};
    float awt=0,atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
    }
    while(done<n) {
        idx=-1;
        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;
        if(idx==-1) {
            time++;
            continue;
        }
        printf("| P%d ",pid[idx]);
        time += bt[idx];
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

SLIP 23

Q.1

Implement the C program to accept n integers to be sorted. Main creates child using fork. Parent sorts using bubble sort and waits using wait. Child sorts using insertion sort.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void bubble(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j]>a[j+1]) {
                t=a[j]; a[j]=a[j+1]; a[j+1]=t;
            }
}
void insertion(int a[], int n)
{
    int i,j,key;
    for(i=1;i<n;i++) {
        key=a[i];
        j=i-1;
        while(j>=0 && a[j]>key) {
            a[j+1]=a[j];
            j--;
        }
        a[j+1]=key;
    }
}
void display(int a[], int n)
{
    int i;
    for(i=0;i<n;i++)
        printf("%d ",a[i]);
    printf("\n");
}
int main()
{
    int n,i;
    int a[50],b[50];
    pid_t pid;
    printf("Enter number of integers: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        scanf("%d",&a[i]);
        b[i]=a[i];
    }
    pid=fork();
    if(pid<0) {
        perror("fork");
        return 1;
    }
    if(pid==0) {
        insertion(b,n);
        printf("Child sorted using insertion sort: ");
        display(b,n);
    } else {
        bubble(a,n);
        printf("Parent sorted using bubble sort: ");
        display(a,n);
        wait(NULL);
    }
    return 0;
}
```

Q.2

Write the program to simulate Non-preemptive Shortest Job First (SJF) scheduling. Accept arrival time and first CPU burst; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time=0, done=0, idx, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20], used[20]={0};
    float awt=0, atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    printf("\nGantt Chart:\n");
    while(done<n) {
        idx=-1;
        for(i=0;i<n;i++)
            if(!used[i] && at[i]<=time)
                if(idx==-1 || bt[i]<bt[idx])
                    idx=i;
        if(idx==-1) {
            time++;
            continue;
        }
        printf("| P%d ",pid[idx]);
        time += bt[idx];
```

```
        ct[idx]=time;
        tat[idx]=ct[idx]-at[idx];
        wt[idx]=tat[idx]-bt[idx];
        used[idx]=1;
        done++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

SLIP 24

Q.1

Write the program to simulate Round Robin (RR) scheduling. Accept arrival time, first CPU burst and time quantum; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n,tq,i,j,time=0,completed=0;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];
    float awt=0,atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d",&tq);
    while(completed<n) {
        int executed=0;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                executed=1;
                printf("| P%d ",pid[i]);
                if(rem[i]>tq) {
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    rem[i]=0;
                    ct[i]=time;
                    tat[i]=ct[i]-at[i];
                    wt[i]=tat[i]-bt[i];
                    completed++;
                }
            }
        }
        if(!executed)
            time++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

Q.2

Implement the C program that accepts an integer array. Main forks child. Parent sorts and passes the sorted array to child through execve() command-line arguments; child loads a new program to perform binary search.

```
/* parent.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,i,key;
    int a[50];
    char *args[60];
    printf("Enter number of integers: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter item to search: ");
    scanf("%d",&key);
    sort(a,n);
    args[0]="./child";
    for(i=0;i<n;i++)
        args[i+1]=(char*)malloc(12);
    for(i=0;i<n;i++)
        sprintf(args[i+1],"%d",a[i]);
    args[n+1]=(char*)malloc(12);
    sprintf(args[n+1],"%d",key);
    args[n+2]=NULL;
    if(fork()==0) {
        execve("./child",args,NULL);
        perror("execve");
```

```
        exit(1);
    }
    wait(NULL);
    return 0;
}
/* child.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[])
{
    int n=argc-2, key=atoi(argv[argc-1]);
    int low=0, high=n-1, mid, i;
    while(low<=high) {
        mid=(low+high)/2;
        if(atoi(argv[mid+1])==key) {
            printf("Item found.\n");
            return 0;
        }
        if(atoi(argv[mid+1])<key)
            low=mid+1;
        else
            high=mid-1;
    }
    printf("Item not found.\n");
    return 0;

}
```

SLIP 25

Q.1

Write a C program to illustrate orphan process. Parent creates a child and terminates before child finishes. Use fork(), sleep(), getpid(), getppid().

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main()
{
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        return 1;
    }
    if(pid == 0) {
        printf("Child PID = %d\n",getpid());
        printf("Initial Parent PID = %d\n",getppid());
        sleep(5);
        printf("Parent PID after parent termination = %d\n",getppid());
    } else {
        printf("Parent PID = %d\n",getpid());
        sleep(1);
        printf("Parent terminating...\n");
        exit(0);
    }
    return 0;
}
```

Q.2

Write a C program that behaves like a shell displaying '$'. Tokenize commands and execute them using a child process. Implement search f filename pattern, search a filename pattern and search c filename pattern.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    char cmd[200], *args[20];
    int i;
    while(1) {
        printf("$ ");
        fgets(cmd,sizeof(cmd),stdin);
        cmd[strcspn(cmd,"\n")]=0;
        if(strcmp(cmd,"exit")==0)
            break;
        args[0]=strtok(cmd," ");
        i=0;
        while(args[i]!=NULL && i<19)
            args[++i]=strtok(NULL," ");
        if(args[0] && strcmp(args[0],"search")==0 && i>=3) {
            FILE *fp=fopen(args[2],"r");
            char line[500];
            int count=0, lineNo=0;
            if(fp==NULL) {
                printf("File cannot be opened.\n");
                continue;
            }
            while(fgets(line,sizeof(line),fp)) {
                lineNo++;
                char *p=line;
                while((p=strstr(p,args[3]))!=NULL) {
                    count++;
                    if(strcmp(args[1],"f")==0) {
                        printf("First occurrence at line %d\n",lineNo);
                        fclose(fp);
                        fp=NULL;
                        break;
                    }
                    if(strcmp(args[1],"a")==0)
                        printf("Occurrence at line %d\n",lineNo);
                    p += strlen(args[3]);
                }
                if(fp==NULL) break;
            }
            if(fp) fclose(fp);
            if(strcmp(args[1],"c")==0)
                printf("Occurrences = %d\n",count);
        } else {
            pid_t pid=fork();
            if(pid==0) {
                execvp(args[0],args);
                perror("Command failed");
                exit(1);
            } else if(pid>0)
                wait(NULL);
        }
    }
    return 0;
}
```

SLIP 26

Q.1

Write the program to simulate Preemptive Priority scheduling. Accept arrival time, first CPU burst and priority for n processes; generate the next CPU burst.

```
#include <stdio.h>
int main()
{
    int n,i,time=0,completed=0,idx;
    int pid[20],at[20],bt[20],pr[20],rem[20],ct[20],tat[20],wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT, BT and Priority for P%d: ",pid[i]);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
        rem[i]=bt[i];
    }
    while(completed<n) {
        idx=-1;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                if(idx==-1 || pr[i]<pr[idx])
                    idx=i;
            }
        }
        if(idx==-1) {
            time++;
            continue;
        }
        rem[idx]--;
        time++;
        if(rem[idx]==0) {
            ct[idx]=time;
            tat[idx]=ct[idx]-at[idx];
            wt[idx]=tat[idx]-bt[idx];
            completed++;
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

Q.2

Write the simulation program for demand paging and show page scheduling and total page faults according to FIFO page replacement. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20];
    int n, nf, i, j, pos = 0;
    int hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++)
        f[i] = -1;
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++)
            if(f[j] == p[i])
                hit = 1;
        if(!hit) {
            f[pos] = p[i];
            pos = (pos + 1) % nf;
            fault++;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 27

Q.1

Write a simulation program for disk scheduling using SSTF algorithm. Accept disk blocks, request string and current head position. Display service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n,i,j,head,total=0,index,diff,min;
    int req[100],done[100]={0};
    printf("Enter number of disk requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter current head position: ");
    scanf("%d",&head);
    printf("\nService order: ");
    for(i=0;i<n;i++) {
        min=100000;
        index=-1;
        for(j=0;j<n;j++) {
            if(!done[j]) {
                diff=abs(head-req[j]);
                if(diff<min) {
                    min=diff;
                    index=j;
                }
            }
        }
        done[index]=1;
        total += abs(head-req[index]);
        head=req[index];
        printf("%d ",head);
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

Q.2

Write the simulation program to implement demand paging and show page scheduling and total page faults according to LRU using counter method. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], counter[20];
    int n, nf, i, j, pos, min, hit, fault = 0;
    int time = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        counter[i] = 0;
    }
    for(i=0;i<n;i++) {
        time++;
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                hit = 1;
                counter[j] = time;
                break;
            }
        }
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                min = counter[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(counter[j] < min) {
                        min = counter[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            counter[pos] = time;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
```

```
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 28

Q.1

Write a simulation program for disk scheduling using SCAN algorithm. Accept disk blocks, request string, current head position and direction. Display service order and total head movements.

```
#include <stdio.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,size,head,i,split,total=0;
    int req[100];
    printf("Enter disk size: ");
    scanf("%d",&size);
    printf("Enter number of requests: ");
    scanf("%d",&n);
    printf("Enter request string: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);
    printf("Enter head position: ");
    scanf("%d",&head);
    sort(req,n);
    printf("Enter direction (0=Left, 1=Right): ");
    scanf("%d",&split);
    printf("\nService order: ");
    if(split==1) {
        for(i=0;i<n;i++)
            if(req[i]>=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += abs(head-(size-1));
        head=size-1;
        for(i=n-1;i>=0;i--)
            if(req[i]<head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    } else {
        for(i=n-1;i>=0;i--)
            if(req[i]<=head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
        total += abs(head-0);
        head=0;
        for(i=0;i<n;i++)
            if(req[i]>head) {
                total += abs(head-req[i]);
                head=req[i];
                printf("%d ",head);
            }
    }
    printf("\nTotal head movements = %d",total);
    return 0;
}
```

Q.2

Write the simulation program for demand paging and show page scheduling and total page faults according to MFU page replacement. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], freq[20];
    int n, nf, i, j, pos, max, hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        freq[i] = 0;
    }
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                freq[j]++;
                hit = 1;
                break;
            }
        }
```

```
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                max = freq[0];
                pos = 0;
                for(j=1;j<nf;j++) {
                    if(freq[j] > max) {
                        max = freq[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            freq[pos] = 1;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```

SLIP 29

Q.1

Implement the C program that accepts an integer array. Main forks child. Parent sorts and passes sorted array to child through execve() command-line arguments; child loads a new program to perform binary search.

```
/* parent.c */
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
void sort(int a[], int n)
{
    int i,j,t;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(a[i]>a[j]) {
                t=a[i]; a[i]=a[j]; a[j]=t;
            }
}
int main()
{
    int n,i,key;
    int a[50];
    char *args[60];
    printf("Enter number of integers: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf("Enter item to search: ");
    scanf("%d",&key);
    sort(a,n);
    args[0]="./child";
    for(i=0;i<n;i++)
        args[i+1]=(char*)malloc(12);
    for(i=0;i<n;i++)
        sprintf(args[i+1],"%d",a[i]);
    args[n+1]=(char*)malloc(12);
    sprintf(args[n+1],"%d",key);
    args[n+2]=NULL;
    if(fork()==0) {
        execve("./child",args,NULL);
        perror("execve");
        exit(1);
    }
    wait(NULL);
    return 0;
}
/* child.c */
#include <stdio.h>
#include <stdlib.h>
int main(int argc,char *argv[])
{
    int n=argc-2, key=atoi(argv[argc-1]);
    int low=0, high=n-1, mid, i;
    while(low<=high) {
        mid=(low+high)/2;
        if(atoi(argv[mid+1])==key) {
            printf("Item found.\n");
            return 0;
        }
        if(atoi(argv[mid+1])<key)
            low=mid+1;
        else
            high=mid-1;
    }
    printf("Item not found.\n");
    return 0;

}
```

Q.2

Write the program to simulate FCFS CPU scheduling. Accept arrival time and first CPU burst for n processes; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n, i, j, time = 0, temp;
    int pid[20], at[20], bt[20], ct[20], tat[20], wt[20];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i] = i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
    }
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(at[i] > at[j]) {
                temp=at[i]; at[i]=at[j]; at[j]=temp;
                temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
                temp=pid[i]; pid[i]=pid[j]; pid[j]=temp;
            }
    printf("\nGantt Chart:\n");
    for(i=0;i<n;i++) {
        if(time < at[i])
            time = at[i];
```

```
        printf("| P%d ",pid[i]);
        time += bt[i];
        ct[i] = time;
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    return 0;
}
```

SLIP 30

Q.1

Write the program to simulate Round Robin (RR) CPU scheduling. Accept arrival time, first CPU burst and time quantum. Assume fixed I/O waiting time of 2 units; generate the next CPU burst randomly. Display Gantt chart, turnaround time, waiting time and averages.

```
#include <stdio.h>
int main()
{
    int n,tq,i,j,time=0,completed=0;
    int pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20];
    float awt=0,atat=0;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        pid[i]=i+1;
        printf("Enter AT and BT for P%d: ",pid[i]);
        scanf("%d%d",&at[i],&bt[i]);
        rem[i]=bt[i];
    }
    printf("Enter Time Quantum: ");
    scanf("%d",&tq);
    while(completed<n) {
        int executed=0;
        for(i=0;i<n;i++) {
            if(at[i]<=time && rem[i]>0) {
                executed=1;
                printf("| P%d ",pid[i]);
                if(rem[i]>tq) {
                    time += tq;
                    rem[i] -= tq;
                } else {
                    time += rem[i];
                    rem[i]=0;
                    ct[i]=time;
                    tat[i]=ct[i]-at[i];
                    wt[i]=tat[i]-bt[i];
                    completed++;
                }
            }
        }
        if(!executed)
            time++;
    }
    printf("|\n");
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(i=0;i<n;i++) {
        awt += wt[i];
        atat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }
    printf("\nAverage Waiting Time = %.2f",awt/n);
    printf("\nAverage Turnaround Time = %.2f",atat/n);
    return 0;
}
```

Q.2

Write the simulation program for demand paging and show page scheduling and total page faults according to MFU page replacement. Assume memory of n frames.

```
#include <stdio.h>
int main()
{
    int p[50], f[20], freq[20];
    int n, nf, i, j, pos, max, hit, fault = 0;
    printf("Enter number of pages: ");
    scanf("%d",&n);
    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&p[i]);
    printf("Enter number of frames: ");
    scanf("%d",&nf);
    for(i=0;i<nf;i++) {
        f[i] = -1;
        freq[i] = 0;
    }
    for(i=0;i<n;i++) {
        hit = 0;
        for(j=0;j<nf;j++) {
            if(f[j] == p[i]) {
                freq[j]++;
                hit = 1;
                break;
            }
        }
        if(!hit) {
            fault++;
            pos = -1;
            for(j=0;j<nf;j++) {
                if(f[j] == -1) {
                    pos = j;
                    break;
                }
            }
            if(pos == -1) {
                max = freq[0];
                pos = 0;
```

```
                for(j=1;j<nf;j++) {
                    if(freq[j] > max) {
                        max = freq[j];
                        pos = j;
                    }
                }
            }
            f[pos] = p[i];
            freq[pos] = 1;
        }
        printf("\n%d : ",p[i]);
        for(j=0;j<nf;j++)
            printf("%d ",f[j]);
    }
    printf("\nPage Faults = %d",fault);
    return 0;
}
```